



**UNIVERSIDAD SIMÓN BOLÍVAR**  
**DECANATO DE ESTUDIOS PROFESIONALES**  
**COORDINACIÓN DE INGENIERÍA ELECTRÓNICA**

**PROTOTIPADO Y EVALUACIÓN DE ENTORNOS DE DESARROLLO  
ASISTIDOS POR LLM CON RAG Y MCP PARA PIC18F47Q10**

Por:

Gerónimo Alejandro Velasco Castellanos

**INFORME DE PASANTÍA**

Presentado ante la Ilustre Universidad Simón Bolívar  
como requisito parcial para optar al título de

Ingeniero Electrónico

**Sartenejas, Febrero de 2027**



**UNIVERSIDAD SIMÓN BOLÍVAR**  
**DECANATO DE ESTUDIOS PROFESIONALES**  
**COORDINACIÓN DE INGENIERÍA ELECTRÓNICA**

**PROTOTIPADO Y EVALUACIÓN DE ENTORNOS DE DESARROLLO  
ASISTIDOS POR LLM CON RAG Y MCP PARA PIC18F47Q10**

Por:

Gerónimo Alejandro Velasco Castellanos

Realizado con la asesoría de:

Tutor Académico: Prof. Oswaldo Monasterios Carrero

Tutor Industrial: Ángel David Rodríguez Niño

**INFORME DE PASANTÍA**

Presentado ante la Ilustre Universidad Simón Bolívar

como requisito parcial para optar al título de

Ingeniero Electrónico

**Sartenejas, Febrero de 2027**

*[Espacio reservado para el Acta de Evaluación de Pasantía,  
con las firmas originales del jurado.]*

## Resumen

**Palabras clave:** PIC18F47Q10, generación de código asistida por inteligencia artificial, generación aumentada por recuperación, modelos de lenguaje de gran escala, sistemas embebidos.

## ÍNDICE GENERAL

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| Resumen                                                            | iv       |
| Índice de tablas                                                   | viii     |
| Índice de figuras                                                  | ix       |
| Lista de símbolos                                                  | x        |
| Lista de abreviaturas                                              | xi       |
| Introducción                                                       | 1        |
| <b>1. Descripción de la empresa</b>                                | <b>6</b> |
| 1.1. Identificación y ubicación . . . . .                          | 6        |
| 1.2. Actividad y servicios . . . . .                               | 6        |
| 1.3. Modelo de trabajo . . . . .                                   | 7        |
| 1.4. Ámbito de la pasantía . . . . .                               | 8        |
| <b>2. Marco teórico</b>                                            | <b>9</b> |
| 2.1. Microcontroladores de 8 bits y la familia PIC18-Q10 . . . . . | 9        |
| 2.1.1. Arquitectura del núcleo . . . . .                           | 9        |
| 2.1.2. Ciclo de instrucción y segmentación . . . . .               | 10       |
| 2.1.3. Organización de la memoria . . . . .                        | 10       |
| 2.1.4. Modos de direccionamiento . . . . .                         | 11       |
| 2.1.5. Palabras de configuración . . . . .                         | 12       |
| 2.1.6. Sistema de interrupciones . . . . .                         | 12       |

|         |                                                                       |    |
|---------|-----------------------------------------------------------------------|----|
| 2.1.7.  | Periféricos independientes del núcleo . . . . .                       | 12 |
| 2.1.8.  | Periféricos disponibles . . . . .                                     | 13 |
| 2.1.9.  | La configuración de un periférico como problema . . . . .             | 14 |
| 2.1.10. | La documentación del fabricante como fuente de conocimiento . . . . . | 14 |
| 2.2.    | Desarrollo de firmware en C con XC8 . . . . .                         | 15 |
| 2.2.1.  | Modelo de compilación . . . . .                                       | 15 |
| 2.2.2.  | Gestión de la memoria bancada . . . . .                               | 16 |
| 2.2.3.  | Generación de configuraciones con MCC . . . . .                       | 16 |
| 2.2.4.  | Criterios de eficiencia . . . . .                                     | 17 |
| 2.3.    | Modelos de lenguaje de gran escala y generación de código . . . . .   | 17 |
| 2.3.1.  | La arquitectura Transformer . . . . .                                 | 17 |
| 2.3.2.  | Tokenización y ventana de contexto . . . . .                          | 18 |
| 2.3.3.  | Estrategias de decodificación . . . . .                               | 18 |
| 2.3.4.  | Generación de código y su evaluación . . . . .                        | 19 |
| 2.3.5.  | Instrucciones de sistema . . . . .                                    | 19 |
| 2.3.6.  | Limitaciones . . . . .                                                | 19 |
| 2.3.7.  | Antecedentes en sistemas embebidos . . . . .                          | 20 |
| 2.4.    | Generación aumentada por recuperación . . . . .                       | 20 |
| 2.4.1.  | Fundamento . . . . .                                                  | 20 |
| 2.4.2.  | Arquitectura . . . . .                                                | 21 |
| 2.4.3.  | Extracción de documentación técnica . . . . .                         | 21 |
| 2.4.4.  | Segmentación del corpus . . . . .                                     | 22 |
| 2.4.5.  | Metadatos y filtrado . . . . .                                        | 22 |
| 2.4.6.  | Representación vectorial y búsqueda densa . . . . .                   | 22 |
| 2.4.7.  | Recuperación cruzada entre idiomas . . . . .                          | 23 |
| 2.4.8.  | Búsqueda léxica y estrategias híbridas . . . . .                      | 24 |
| 2.4.9.  | Índices aproximados . . . . .                                         | 24 |
| 2.4.10. | Bases de datos vectoriales . . . . .                                  | 24 |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| 2.4.11. Reordenamiento . . . . .                                         | 25        |
| 2.4.12. Construcción del contexto . . . . .                              | 25        |
| 2.4.13. Recuperación frente a ajuste fino . . . . .                      | 25        |
| 2.4.14. Variantes . . . . .                                              | 26        |
| 2.5. Exposición del sistema mediante el Model Context Protocol . . . . . | 26        |
| 2.5.1. Arquitectura y capas . . . . .                                    | 27        |
| 2.5.2. Primitivas y control . . . . .                                    | 27        |
| 2.5.3. Estado de la especificación . . . . .                             | 27        |
| 2.6. Evaluación de sistemas de generación de código . . . . .            | 28        |
| 2.6.1. Métricas sobre el código generado . . . . .                       | 28        |
| 2.6.2. Evaluación del recuperador . . . . .                              | 28        |
| 2.6.3. Validación automática con realimentación . . . . .                | 29        |
| 2.6.4. Diseño experimental . . . . .                                     | 29        |
| <b>3. Metodología</b>                                                    | <b>31</b> |
| <b>4. Resultados y discusión</b>                                         | <b>32</b> |
| <b>Conclusiones y recomendaciones</b>                                    | <b>33</b> |
| <b>Referencias</b>                                                       | <b>34</b> |

## ÍNDICE DE TABLAS

|                                                             |    |
|-------------------------------------------------------------|----|
| 1.1. Líneas de servicio de Domotai C.A. . . . .             | 7  |
| 2.1. Recursos de memoria del PIC18F47Q10 [1] . . . . .      | 11 |
| 2.2. Periféricos del PIC18F47Q10 [1] . . . . .              | 13 |
| 2.3. Recuperación aumentada frente a ajuste fino . . . . .  | 26 |
| 2.4. Métricas de evaluación y componente evaluado . . . . . | 30 |

## ÍNDICE DE FIGURAS

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| 2.1. Reparto de la memoria de datos del PIC18F47Q10 en bancos [1] . . . . .  | 11 |
| 2.2. Etapas de un sistema de generación aumentada por recuperación . . . . . | 21 |
| 2.3. Ciclo de validación con realimentación de errores . . . . .             | 29 |

## Lista de símbolos

### Símbolos latinos

|          |                                                                        |
|----------|------------------------------------------------------------------------|
| $f$      | Frecuencia de operación del microcontrolador [Hz]                      |
| $c$      | Número de muestras correctas de un problema                            |
| $d_k$    | Dimensión de las claves en el mecanismo de atención                    |
| $k$      | Número de fragmentos recuperados por consulta, o de muestras generadas |
| $N$      | Número de consultas del conjunto de evaluación                         |
| $n$      | Número de muestras generadas por problema                              |
| $V_{DD}$ | Tensión de alimentación [V]                                            |

### Símbolos griegos

|        |                                                             |
|--------|-------------------------------------------------------------|
| $\tau$ | Tiempo de adquisición del convertidor analógico-digital [s] |
|--------|-------------------------------------------------------------|

## Lista de abreviaturas

| Sigla            | Significado                                                                           |
|------------------|---------------------------------------------------------------------------------------|
| ADC              | <i>Analog-to-Digital Converter</i> — convertidor analógico-digital                    |
| ADCC             | <i>ADC with Computation</i> — ADC con cómputo integrado                               |
| BM25             | <i>Best Matching 25</i> — función de puntuación para recuperación léxica              |
| BSR              | <i>Bank Select Register</i> — registro de selección de banco                          |
| CCP              | <i>Capture/Compare/PWM</i>                                                            |
| CLC              | <i>Configurable Logic Cell</i> — celda lógica configurable                            |
| CVD              | <i>Capacitive Voltage Divider</i> — divisor de tensión capacitivo                     |
| CWG              | <i>Complementary Waveform Generator</i>                                               |
| DAC              | <i>Digital-to-Analog Converter</i> — convertidor digital-analógico                    |
| DFM              | <i>Data Flash Memory</i> — memoria Flash de datos (EEPROM)                            |
| DPR              | <i>Dense Passage Retrieval</i> — recuperación densa de pasajes                        |
| DSM              | <i>Data Signal Modulator</i> — modulador de señal de datos                            |
| EEPROM           | <i>Electrically Erasable Programmable Read-Only Memory</i>                            |
| EUSART           | <i>Enhanced Universal Synchronous Asynchronous Receiver Transmitter</i>               |
| FVR              | <i>Fixed Voltage Reference</i> — referencia de tensión fija                           |
| GPIO             | <i>General Purpose Input/Output</i> — entrada/salida de propósito general             |
| GPR              | <i>General Purpose Register</i> — registro de propósito general                       |
| HLT              | <i>Hardware Limit Timer</i>                                                           |
| HLVD             | <i>High/Low-Voltage Detect</i> — detección de alta y baja tensión                     |
| HNSW             | <i>Hierarchical Navigable Small World</i> — grafo jerárquico para búsqueda aproximada |
| ICSP             | <i>In-Circuit Serial Programming</i>                                                  |
| IDE              | <i>Integrated Development Environment</i> — entorno de desarrollo integrado           |
| I <sup>2</sup> C | <i>Inter-Integrated Circuit</i>                                                       |
| LLM              | <i>Large Language Model</i> — modelo de lenguaje de gran escala                       |
| MCC              | <i>MPLAB Code Configurator</i>                                                        |
| MCU              | <i>Microcontroller Unit</i> — microcontrolador                                        |

| Sigla | Significado                                                                   |
|-------|-------------------------------------------------------------------------------|
| MRR   | <i>Mean Reciprocal Rank</i> — rango recíproco medio                           |
| MSSP  | <i>Master Synchronous Serial Port</i>                                         |
| NLP   | <i>Natural Language Processing</i> — procesamiento de lenguaje natural        |
| PFM   | <i>Program Flash Memory</i> — memoria Flash de programa                       |
| PMD   | <i>Peripheral Module Disable</i> — deshabilitación de módulos periféricos     |
| PPS   | <i>Peripheral Pin Select</i> — selección de pines de periférico               |
| PWM   | <i>Pulse-Width Modulation</i> — modulación por ancho de pulso                 |
| RAG   | <i>Retrieval-Augmented Generation</i> — generación aumentada por recuperación |
| SFR   | <i>Special Function Register</i> — registro de función especial               |
| SPI   | <i>Serial Peripheral Interface</i>                                            |
| SRAM  | <i>Static Random-Access Memory</i>                                            |
| USB   | Universidad Simón Bolívar                                                     |
| WWDT  | <i>Windowed Watchdog Timer</i> — temporizador de guardia con ventana          |
| ZCD   | <i>Zero-Cross Detection</i> — detección de cruce por cero                     |

## Introducción

El desarrollo de firmware para microcontroladores de 8 bits exige un conocimiento detallado de la hoja de datos del dispositivo, la realización de proyectos de prueba que permitan comprender la configuración de cada periférico y el dominio de un entorno de desarrollo específico para cada familia empleada. Ese conocimiento no es transferible entre fabricantes ni, en muchos casos, entre familias de un mismo fabricante, de modo que cada nuevo dispositivo impone un período de aprendizaje antes de que resulte productivo.

El presente trabajo, realizado como pasantía larga en la empresa Domotai C.A., aborda ese coste mediante un módulo de generación de código asistida por inteligencia artificial para el microcontrolador PIC18F47Q10 de Microchip Technology.

## Antecedentes

Los modelos de lenguaje de gran escala, designados habitualmente por sus siglas en inglés LLM (*Large Language Model*), han alcanzado en los últimos años una capacidad notable para producir código a partir de descripciones en lenguaje natural. El trabajo que estableció el marco de evaluación hoy dominante presentó un modelo entrenado sobre código junto con un conjunto de problemas de programación y la métrica *pass@k*, que estima la probabilidad de que al menos una de  $k$  generaciones supere las pruebas asociadas [2].

Esa capacidad se ha caracterizado también en el dominio de los sistemas empujados. Un estudio experimental sobre desarrollo y depuración concluye que los modelos actuales generan controladores a nivel de registro y código de comunicación funcional, y que incluso cuando el resultado no es correcto el razonamiento que lo acompaña resulta de utilidad para el desarrollador [3]. El mismo trabajo aporta dos observaciones determinantes para esta pasantía: parte del conocimiento del modelo sobre el hardware procede de la presencia de hojas de datos en su corpus de entrenamiento, y suministrarle una representación legible de los registros pertinentes mejora sustancialmente el resultado.

En paralelo, la técnica de generación aumentada por recuperación, conocida por sus siglas en inglés RAG (*Retrieval-Augmented Generation*), ha demostrado que combinar un

modelo de lenguaje con un sistema de recuperación sobre un corpus externo permite anclar las respuestas a documentos concretos y actualizarlas sin reentrenar el modelo [4]. Su aplicación a la generación de firmware ya ha sido explorada: un trabajo reciente la emplea para producir capas de abstracción de hardware para microcontroladores de otro fabricante, integrándola en el flujo de desarrollo existente [5].

Existe, por tanto, evidencia de que la técnica es viable y de que el problema concreto —la fidelidad a los registros de un dispositivo— es precisamente el que la recuperación puede resolver. Lo que no consta es su realización sobre la familia PIC18-Q10 de Microchip y su cadena de herramientas, ni un ciclo de validación que verifique automáticamente el código producido.

## **Justificación e importancia**

La utilidad de un asistente de generación de código para firmware depende por completo de la exactitud de lo que produce. Un nombre de registro inexistente, un campo de bits con anchura equivocada o la omisión de un paso de inicialización no generan una respuesta aproximada, sino un programa que no compila o, peor, uno que compila y no funciona. En este dominio no existe la respuesta parcialmente correcta.

Los modelos de lenguaje presentan tres limitaciones que agravan esa exigencia. La primera es la generación de contenido incorrecto con apariencia plausible. La segunda es el corte de conocimiento: el modelo solo dispone de lo que estaba en su corpus de entrenamiento, sin acceso a documentación posterior ni a las erratas que el fabricante publica para corregir sus propios manuales. La tercera, específica de este ámbito, es la escasez de vocabulario especializado: los nombres de registro y las constantes de hardware aparecen con muy baja frecuencia en corpus generalistas, de modo que el modelo dispone de poca evidencia estadística sobre ellos.

La recuperación aumentada aborda las tres simultáneamente. El conocimiento del dispositivo deja de residir en los parámetros del modelo y pasa a un índice consultable: se actualiza reindexando, la respuesta queda anclada a un fragmento verificable y los identificadores exactos llegan al modelo en el propio contexto. A ello se añade una ventaja que resulta decisiva en documentación técnica: la jerarquía entre fuentes puede imponerse de forma explícita al recuperar, de manera que la errata de silicio prevalezca sobre el manual al que corrige [6]. Un modelo ajustado sobre ambos documentos los mezclaría en sus pesos sin conservar esa relación de precedencia.

Para Domotai C.A., la relevancia es directa: reducir el tiempo que media entre la elección de un microcontrolador y la obtención de firmware funcional, conservando la trazabilidad de

cada decisión de configuración hasta la sección del manual que la respalda.

## Planteamiento del problema

Configurar un periférico en la familia PIC18-Q10 no se agota en escribir los registros del propio módulo. Intervienen al menos tres mecanismos transversales que actúan sobre él y que se documentan en secciones alejadas del manual: la selección de pines de periférico, que desacopla las señales digitales de los pines físicos y exige una secuencia de desbloqueo; la deshabilitación selectiva de módulos, que hace que un periférico no responda a ninguna escritura; y el bit que habilita el juego de instrucciones extendido, que altera la semántica del direccionamiento de datos para todo el programa. A ellos se suma el direccionamiento por bancos de la memoria de datos.

La consecuencia es que un fragmento de código correcto según la sección del periférico puede fallar por causas descritas doscientas páginas más adelante. La información necesaria para escribir una inicialización completa está distribuida, no concentrada.

Esa distribución convive con un segundo obstáculo. La hoja de datos del dispositivo supera las setecientas páginas [1], de modo que no cabe íntegra en la ventana de contexto de los modelos de uso corriente; y aun cuando cupiera, la evidencia experimental muestra que el aprovechamiento del contexto se degrada con su longitud, y que los elementos situados en posiciones centrales se utilizan peor que los de los extremos [7]. Suministrar el manual completo no es, por tanto, una alternativa viable a recuperar selectivamente lo pertinente.

A ello se añade que la información determinante —mapas de registros, asignación de pines, mapas de memoria— reside en tablas y figuras, no en prosa continua, lo que condiciona tanto la extracción del documento como su segmentación.

El problema que este trabajo aborda es, en consecuencia, el de construir un sistema capaz de recuperar de esa documentación los fragmentos pertinentes a una petición concreta, entregárselos a un modelo de lenguaje y verificar automáticamente que el código resultante es correcto para el dispositivo.

## Objetivo general

Desarrollar un módulo de generación de código asistido por un modelo de lenguaje de gran escala que, mediante un sistema de generación aumentada por recuperación, produzca código verificable para el microcontrolador de 8 bits PIC18F47Q10 de Microchip, integrando su hoja de datos, la documentación de los periféricos y un conjunto de configuraciones base validadas.

## Objetivos específicos

1. **Revisar el estado del arte.** Analizar las técnicas actuales de generación de código asistida por inteligencia artificial y de generación aumentada por recuperación, identificando sus capacidades, sus limitaciones y las necesidades del problema a resolver.
2. **Configurar el entorno de desarrollo.** Establecer la cadena de herramientas para el PIC18F47Q10 —el entorno de desarrollo de Microchip, el compilador XC8 y el configurador de código MCC—, adquirir el manejo de la tarjeta de evaluación correspondiente y definir los criterios de eficiencia en el uso de la memoria de programa y de datos.
3. **Generar las configuraciones y ejemplos base.** Desarrollar, compilar y validar en hardware un conjunto de ejemplos de referencia para los periféricos del dispositivo: pines de entrada y salida de propósito general, temporizadores, modulación por ancho de pulso, convertidor analógico-digital, convertidor digital-analógico, comparadores y las interfaces de comunicación EUSART, I<sup>2</sup>C y SPI. El proceso se acompañará de la documentación de los registros y las restricciones de cada periférico.
4. **Construir la base de conocimiento y el sistema de recuperación.** Procesar y segmentar la hoja de datos y la documentación de los periféricos, e indexar los ejemplos validados, de forma que se permita la recuperación precisa de la información técnica pertinente.
5. **Desarrollar el módulo de generación.** Integrar el modelo de lenguaje con el sistema de recuperación, diseñando las instrucciones de sistema y el flujo que conduzca a una generación de código efectiva.
6. **Evaluar la validación automática del código.** Establecer un mecanismo de compilación, verificación y realimentación del código generado, incorporando un ciclo que reinyecte los errores del compilador al modelo para su corrección iterativa.
7. **Evaluar el desempeño y elaborar la documentación técnica.** Definir y medir métricas de calidad —tasa de compilación exitosa, fidelidad a las configuraciones base y eficiencia en recursos—, comparando el desempeño del sistema con y sin recuperación, y generar la documentación con la arquitectura, los resultados y las especificaciones del módulo.

## Estructura del informe

El documento se organiza en cuatro capítulos. El primero describe la empresa donde se desarrolló la pasantía. El segundo reúne el marco teórico, recorriendo desde la arquitectura del

microcontrolador y su cadena de herramientas hasta los modelos de lenguaje, la generación aumentada por recuperación y los criterios de evaluación. El tercero expone la metodología seguida, y el cuarto presenta los resultados obtenidos junto con su discusión. Cierran el documento las conclusiones y recomendaciones, las referencias consultadas y los apéndices.

# **CAPÍTULO 1**

## **DESCRIPCIÓN DE LA EMPRESA**

La pasantía se desarrolla en Domotai C.A., una empresa venezolana de desarrollo de software cuya actividad se concentra en la aplicación de la inteligencia artificial a productos y procesos. Este capítulo describe su actividad, sus líneas de servicio y su forma de trabajo, y sitúa el proyecto de la pasantía dentro de ese contexto.

### **1.1. Identificación y ubicación**

La empresa opera bajo la razón social Domotai C.A. y tiene su sede en el Edificio Caracas Campus, calle Altagracia, urbanización La Trinidad, municipio Baruta del estado Miranda, en la ciudad de Caracas.

Su ámbito de operación no se limita al mercado local: los proyectos referidos en su portafolio corresponden a clientes de sectores diversos, atendidos bajo un modelo de trabajo que no exige presencia física del cliente.

### **1.2. Actividad y servicios**

Domotai C.A. se define como una empresa de desarrollo de software que combina el criterio de sus profesionales con herramientas de inteligencia artificial para la construcción de productos. Su oferta abarca las líneas recogidas en la tabla 1.1.

Tabla 1.1. Líneas de servicio de Domotai C.A.

| Línea                                       | Alcance                                                                              |
|---------------------------------------------|--------------------------------------------------------------------------------------|
| Desarrollo web                              | Sitios corporativos y aplicaciones de cliente desarrollados a medida                 |
| Desarrollo móvil                            | Aplicaciones multiplataforma                                                         |
| Automatización con inteligencia artificial  | Agentes y flujos automatizados integrados en los procesos del cliente                |
| Integración <i>hardware-software</i>        | Desarrollo de soluciones que articulan componentes físicos con sistemas informáticos |
| Diseño de interfaz y experiencia de usuario | Definición de la interacción y del aspecto visual de los productos                   |
| Asesoría tecnológica                        | Acompañamiento en decisiones de arquitectura y selección de herramientas             |

De estas líneas, dos resultan directamente pertinentes para el presente trabajo y se retoman al final del capítulo: la automatización mediante inteligencia artificial y la integración entre *hardware* y *software*.

### 1.3. Modelo de trabajo

La empresa acumula más de veinte proyectos completados, con un plazo de entrega característico en torno a las seis semanas. Esa cifra describe un modo de operar orientado a ciclos cortos, en los que el producto se acota para poder entregarse en un plazo acotado en lugar de abordarse como un desarrollo prolongado.

Los proyectos de su portafolio abarcan ámbitos heterogéneos —diseño y arte, hostelería, interiorismo y comercio electrónico—, lo que indica que la especialización de la empresa no reside en un sector vertical concreto sino en un conjunto de capacidades técnicas aplicables a dominios distintos.

La contratación se articula bajo dos modalidades: por proyecto, con un alcance cerrado de antemano, y por suscripción, que da al cliente acceso continuado a la capacidad de desarrollo de la empresa. La segunda modalidad supone una relación sostenida en el tiempo, lo que favorece la acumulación de conocimiento sobre el dominio de cada cliente.

## 1.4. Ámbito de la pasantía

El proyecto desarrollado durante esta pasantía se sitúa en la intersección de dos de las líneas de servicio descritas. Por un lado, se trata de un sistema de automatización basado en un modelo de lenguaje, esto es, de la aplicación de inteligencia artificial a una tarea que hasta ahora exigía trabajo manual especializado. Por otro, su objeto es la generación de firmware para un microcontrolador, de modo que pertenece al terreno de la integración entre *hardware* y *software*.

Esa doble pertenencia explica el encaje del proyecto en la empresa. El desarrollo de firmware exige un conocimiento específico de cada dispositivo que no se transfiere entre familias de microcontroladores, lo que resulta especialmente costoso para una organización que trabaja por ciclos cortos y sobre dominios heterogéneos. Un sistema capaz de producir código verificable a partir de la documentación del fabricante reduce ese coste de entrada y, en consecuencia, hace viable abordar proyectos con componente empotrado sin que el plazo de aprendizaje condicione la planificación.

La supervisión de la pasantía corresponde a Ángel David Rodríguez Niño, Técnico Superior Universitario en Electrónica y Gerente de Tecnología de la compañía, designado tutor industrial. La estancia se desarrolla en modalidad presencial en la sede de la empresa, con una duración de veinte semanas.

## CAPÍTULO 2

### MARCO TEÓRICO

Este capítulo reúne los fundamentos sobre los que se apoya el desarrollo. Avanza desde el dispositivo hacia el sistema de generación: primero se describen el microcontrolador y la cadena de herramientas que producen el código objetivo, después los modelos de lenguaje capaces de escribirlo, la técnica de recuperación que les aporta el conocimiento específico del dispositivo, el protocolo con que ese conocimiento se les expone y, por último, los criterios con que se mide el resultado.

#### 2.1. Microcontroladores de 8 bits y la familia PIC18-Q10

Un microcontrolador integra en un solo circuito el procesador, la memoria y los periféricos necesarios para gobernar un sistema empotrado. Frente a un procesador de propósito general, sacrifica capacidad de cómputo a cambio de determinismo temporal, bajo consumo y coste reducido, lo que lo hace idóneo para tareas de control en tiempo real.

El dispositivo objetivo de este trabajo es el PIC18F47Q10 de Microchip Technology, un microcontrolador de 8 bits de la familia PIC18-Q10 [1].

##### 2.1.1. Arquitectura del núcleo

El núcleo sigue una arquitectura Harvard modificada con juego de instrucciones reducido: la memoria de programa y la de datos ocupan espacios separados, con buses independientes y anchuras distintas —16 bits para las instrucciones y 8 bits para los datos—. Esa separación permite solapar la búsqueda de una instrucción con la ejecución de la anterior.

El repertorio consta de 75 instrucciones, ampliables a 83 si se habilita el juego extendido. La frecuencia de trabajo llega hasta 64 MHz, con un ciclo de instrucción mínimo de 62,5 ns, ya que cada ciclo abarca cuatro períodos del oscilador. El contador de programa es de 21 bits, de modo que direcciona un espacio de 2 MB aunque el dispositivo no lo implemente por completo.

La pila de retorno admite 31 niveles y reside en un panel de memoria estática independiente, fuera de los espacios de programa y de datos. Es accesible por software, lo que permite implementar mecanismos de conmutación de contexto, pero su desbordamiento provoca un reinicio del dispositivo si así lo determina el bit de configuración correspondiente. Las interrupciones admiten dos niveles de prioridad programables.

### **2.1.2. Ciclo de instrucción y segmentación**

Cada ciclo de instrucción abarca cuatro períodos del oscilador, denominados Q1 a Q4. La búsqueda del código de operación ocupa uno de ellos y la ejecución los restantes, de modo que el núcleo solapa la búsqueda de la instrucción siguiente con la ejecución de la actual. En régimen permanente se completa una instrucción por ciclo.

La segmentación se rompe cuando el flujo cambia. Las instrucciones de salto y las condicionales cuya condición se cumple invalidan la instrucción ya buscada, que se sustituye por una operación nula, y consumen dos ciclos. Las cuatro instrucciones que ocupan dos palabras de memoria consumen igualmente dos ciclos. Esta irregularidad es la razón por la que el conteo de ciclos de una rutina no coincide con el número de instrucciones que la componen, dato relevante cuando se generan rutinas de temporización por software.

### **2.1.3. Organización de la memoria**

La memoria de datos se direcciona con 12 bits y se organiza en dieciséis bancos de 256 bytes. Como la mayoría de las instrucciones solo codifica los 8 bits menos significativos de la dirección, los 4 restantes proceden del registro de selección de banco. Este esquema introduce una fuente de error característica de la arquitectura: operar sobre el banco equivocado escribe en una dirección distinta de la prevista, y el registro de estado se actualiza como si la operación hubiera tenido éxito.

Para atenuarlo, la arquitectura reserva un espacio de acceso rápido de 256 bytes, accesible en un solo ciclo sin necesidad de fijar el banco. Lo componen los 96 bytes bajos del banco 0, destinados a variables de uso frecuente, y los 160 bytes altos del banco 15, donde residen los registros de función especial más utilizados.

La tabla 2.1 resume las capacidades del dispositivo.

Tabla 2.1. Recursos de memoria del PIC18F47Q10 [1]

| Recurso             | Capacidad  | Observación                             |
|---------------------|------------|-----------------------------------------|
| Memoria de programa | 131.072 B  | 65.536 instrucciones de 16 bits         |
| Memoria de datos    | 3.615 B    | 3.359 B utilizables más 256 B de sector |
| Memoria EEPROM      | 1.024 B    | Tecnología Flash, escritura byte a byte |
| Pila de retorno     | 31 niveles | Panel independiente de 21 bits          |

La figura 2.1 representa el reparto de los dieciséis bancos en el dispositivo empleado.

|                    |                    |
|--------------------|--------------------|
| Bancos 0–12        | GPR — 3.328 B      |
| Banco 13           | Sector RAM — 256 B |
| Banco 14 (00h–1Eh) | GPR — 31 B         |
| Banco 14 (1Fh–FFh) | SFR                |
| Banco 15           | SFR                |

Figura 2.1. Reparto de la memoria de datos del PIC18F47Q10 en bancos [1]

Los registros de propósito general suman 3.359 bytes utilizables. El banco 13 no almacena variables: aloja los registros intermedios que participan en la escritura de la memoria de programa por sectores.

#### 2.1.4. Modos de direccionamiento

La arquitectura ofrece cuatro modos, y la elección entre ellos determina qué estructuras de datos son expresables con eficiencia.

El direccionamiento **inherente** corresponde a instrucciones sin operando explícito. El **literal** incorpora una constante en la propia instrucción. El **directo** codifica los 8 bits bajos de la dirección y obtiene los 4 restantes del registro de selección de banco, salvo que un bit de la instrucción fuerce el espacio de acceso rápido.

El **indirecto** emplea tres pares de registros de selección de archivo que albergan direcciones completas de 12 bits, con los que se alcanza cualquier banco sin depender del registro de selección. El acceso se realiza a través de operandos virtuales que, además de leer o escribir, incrementan o decrementan automáticamente el puntero, o le aplican un desplazamiento con signo. Son estos operandos los que hacen viables los vectores y las tablas en memoria de datos.

Habilitar el juego de instrucciones extendido añade un quinto modo, indexado con desplazamiento literal, y **altera la interpretación del espacio de acceso rápido**, lo que constituye una de las incompatibilidades más difíciles de diagnosticar en esta familia.

### 2.1.5. Palabras de configuración

Seis palabras de 16 bits, grabadas junto al programa a partir de la dirección 0x300000, fijan el comportamiento del dispositivo desde el arranque [1]. Determinan la fuente de reloj inicial, el comportamiento del temporizador de guardia, los umbrales de reinicio por caída de tensión, la protección de las regiones de memoria y la habilitación del juego extendido.

Su particularidad es que no son modificables en tiempo de ejecución: un valor equivocado no produce un error de compilación sino un dispositivo que arranca a una frecuencia distinta de la prevista, o que se reinicia sin causa aparente. Por tanto, cualquier código de inicialización completo debe incluirlas.

### 2.1.6. Sistema de interrupciones

El dispositivo atiende las interrupciones mediante dos vectores fijos, situados en las direcciones 0x0008 y 0x0018 de la memoria de programa, correspondientes a los niveles de prioridad alta y baja [1]. Cada fuente dispone de un bit de habilitación individual, un bit indicador de que el suceso se ha producido y un bit que fija su prioridad, además de dos bits de habilitación global.

Dos particularidades condicionan el código que atiende una interrupción. La primera es que el indicador no se limpia por hardware al entrar en la rutina: corresponde al programador borrarlo, y omitirlo provoca que la rutina se invoque de forma indefinida. La segunda es la conservación del contexto. La arquitectura dispone de una pila rápida de un solo nivel para tres registros del núcleo, que se guarda automáticamente al vectorizar; pero si se emplean los dos niveles de prioridad, una interrupción de prioridad alta que interrumpa a una de baja sobrescribe esos valores. En ese caso el contexto debe conservarse por software.

Ambas son fuentes de error que no se manifiestan como fallos de compilación sino como comportamiento errático en ejecución, lo que las hace especialmente adecuadas para valorar la corrección del código generado.

### 2.1.7. Periféricos independientes del núcleo

El rasgo distintivo de la familia Q10 es su conjunto de periféricos independientes del núcleo, capaces de operar sin intervención del procesador. El dispositivo integra ocho celdas lógicas configurables, dos módulos de captura, comparación y modulación por ancho de pulso, dos moduladores adicionales de 10 bits, un generador de formas de onda complementarias, cuatro temporizadores de 16 bits y tres de 8 bits con limitador por hardware, dos transmisores-receptores universales síncronos y asíncronos mejorados, y dos puertos serie síncronos maestros

que implementan las interfaces SPI e I<sup>2</sup>C.

En el apartado analógico destaca el convertidor analógico-digital con computación: además de la conversión a 10 bits sobre 35 canales externos, incorpora en hardware el promediado, el filtrado, el sobremuestreo y la comparación contra umbrales. Completan el conjunto dos comparadores, un convertidor digital-analógico de 5 bits, una referencia de tensión fija, un detector de cruce por cero y un detector de tensión alta y baja.

La consecuencia de diseño es relevante para este trabajo: una función que en otras arquitecturas exigiría atención continua del procesador puede resolverse configurando registros, con el núcleo detenido. El precio es que la complejidad se traslada del código a la configuración.

### 2.1.8. Periféricos disponibles

La tabla 2.2 recoge el inventario del dispositivo, que fija el alcance funcional que un sistema generador debe ser capaz de cubrir.

Tabla 2.2. Periféricos del PIC18F47Q10 [1]

| <b>Categoría</b> | <b>Módulo</b>                                 | <b>Cantidad</b> |
|------------------|-----------------------------------------------|-----------------|
| Temporización    | Temporizadores de 16 bits                     | 4               |
|                  | Temporizadores de 8 bits con limitador        | 3               |
|                  | Captura, comparación y modulación             | 2               |
|                  | Moduladores de ancho de pulso                 | 2               |
| Lógica           | Celdas lógicas configurables                  | 8               |
|                  | Generador de formas complementarias           | 1               |
|                  | Modulador de señal de datos                   | 1               |
| Comunicación     | Puertos serie síncronos maestros              | 2               |
|                  | Transmisores-receptores universales           | 2               |
| Analógico        | Convertidor analógico-digital con computación | 1               |
|                  | Comparadores                                  | 2               |
|                  | Convertidor digital-analógico de 5 bits       | 1               |

La memoria de programa admite escritura desde el propio firmware, lo que posibilita cargadores de arranque residentes. Su borrado opera por sectores de 128 palabras, y toda operación de escritura exige una secuencia de desbloqueo que debe ejecutarse sin interrupciones [1].

### 2.1.9. La configuración de un periférico como problema

Poner en marcha un periférico en esta familia no se agota en escribir sus propios registros. Intervienen al menos tres mecanismos transversales que actúan sobre módulos distintos y que se documentan en secciones alejadas del manual.

El primero es la selección de pines de periférico, que desacopla las señales digitales de los pines físicos. Una salida no aparece en un pin hasta que se escribe el registro de asignación correspondiente, y la secuencia de desbloqueo debe respetarse con exactitud. Un bit de configuración puede además limitar ese desbloqueo a una sola vez tras el reinicio, de modo que un segundo intento de remapeo queda sin efecto.

El segundo es la deshabilitación selectiva de módulos, pensada para reducir el consumo. Un periférico cuyo bit correspondiente esté activo no responde a ninguna escritura, y el síntoma —un módulo que se configura sin error y no funciona— no apunta por sí mismo a la causa.

El tercero es el bit que habilita el juego de instrucciones extendido, que altera la semántica del direccionamiento de datos para todo el programa. La práctica totalidad del código de ejemplo disponible presupone que está desactivado, sin declararlo.

A estos se suma el direccionamiento por bancos ya descrito. El resultado es que un fragmento de código correcto según la sección del periférico puede fallar por causas descritas doscientas páginas más adelante.

### 2.1.10. La documentación del fabricante como fuente de conocimiento

La información necesaria para configurar el dispositivo está distribuida en un conjunto de documentos con distinto alcance y distinta autoridad. El manual principal describe el comportamiento nominal [1]; la errata de silicio recoge las desviaciones observadas en el circuito real y las correcciones al manual, de modo que **prevalece sobre él ante cualquier conflicto** [6]; la especificación de programación documenta el acceso de bajo nivel a las memorias no volátiles [8]; y las notas de aplicación y los informes técnicos ilustran usos concretos de cada módulo.

Dos rasgos de este material condicionan el diseño de un sistema de recuperación. El primero es que el manual cubre simultáneamente varios dispositivos de la familia, con tablas que los diferencian: una cifra leída sin comprobar la columna correspondiente puede pertenecer a otro integrado. El segundo es que la información determinante —mapas de registros, asignación de pines, mapas de memoria— reside en tablas y figuras, no en prosa continua.

## 2.2. Desarrollo de firmware en C con XC8

El código que el sistema debe producir se escribe en lenguaje C y se compila con XC8, el compilador de Microchip para sus microcontroladores de 8 bits [9].

### 2.2.1. Modelo de compilación

XC8 implementa el estándar C con extensiones específicas para la arquitectura. La cabecera `<xc.h>` expone los registros de función especial como variables y da acceso a sus campos de bits mediante estructuras, de modo que el código fuente reproduce la nomenclatura del manual. Las directivas `#pragma config` fijan los bits de las palabras de configuración, que se graban junto al programa y determinan el comportamiento del dispositivo desde el arranque.

Las rutinas de atención a interrupción se declaran con calificadores propios que indican su nivel de prioridad. La gestión de la memoria bancada es responsabilidad del compilador, que inserta las instrucciones de selección de banco necesarias, pero el programador conserva control sobre la ubicación de las variables.

La cadena se completa con el entorno de desarrollo integrado y con la placa de evaluación, cuyo depurador integrado permite programar y depurar el dispositivo sin instrumental adicional [10].

Al finalizar la compilación, XC8 emite un informe con el consumo de memoria de programa y de datos. Ese informe es la fuente de los criterios de eficiencia que se emplean más adelante.

El código 2.1 ilustra la estructura de un programa mínimo. Concentra los elementos que todo código generado debe incluir: palabras de configuración, frecuencia del oscilador, inicialización de los registros de dirección y de modo analógico, y bucle principal.

```

1  #include <xc.h>
2
3  // Palabras de configuracion: fijan el comportamiento desde el arranque
4  #pragma config FEXTOSC = OFF      // Sin oscilador externo
5  #pragma config RSTOSC = HFINTOSC_64MHZ
6  #pragma config WDTE = OFF        // Temporizador de guardia deshabilitado
7  #pragma config XINST = OFF       // Juego extendido desactivado
8
9  #define _XTAL_FREQ 64000000UL    // Requerido por las rutinas de retardo
10
11 void main(void)
12 {
13     ANSELDbits.ANSELD0 = 0;      // RDO en modo digital, no analogico
14     TRISDbits.TRISD0 = 0;       // RDO como salida

```

```

15
16     while (1) {
17         LATDbits.LATD0 ^= 1;      // Conmuta el nivel del pin
18         __delay_ms(500);
19     }
20 }

```

Código 2.1. Estructura mínima de un programa para el PIC18F47Q10

Obsérvese que la asignación del modo analógico precede a la de dirección: los pines con capacidad analógica arrancan en ese modo, de manera que omitir esa línea produce un programa que compila y no conmuta el pin. Es un ejemplo del tipo de conocimiento tácito que un sistema generador debe incorporar.

### 2.2.2. Gestión de la memoria bancada

La memoria de datos en bancos, descrita en la sección anterior, plantea al compilador una decisión que el lenguaje C no expresa: dónde ubicar cada variable y cuándo conmutar de banco. XC8 la resuelve automáticamente, insertando las instrucciones de selección necesarias y agrupando las variables para minimizar el número de conmutaciones.

El programador conserva, no obstante, mecanismos para intervenir. Puede solicitar que una variable resida en el espacio de acceso rápido, con lo que se alcanza en un solo ciclo sin conmutar de banco, o fijarla en una dirección concreta. Ambas posibilidades importan cuando se compara la eficiencia de realizaciones distintas de una misma funcionalidad, porque una diferencia en el consumo de memoria de datos puede deberse tanto al algoritmo como a estas decisiones de ubicación.

### 2.2.3. Generación de configuraciones con MCC

El configurador de código de MPLAB, conocido por sus siglas en inglés MCC (*MPLAB Code Configurator*), permite describir gráficamente la configuración de los periféricos y generar a partir de ella el código de inicialización. Guarda esa descripción en un archivo propio con extensión `.mc3`, separado del código que produce [11].

La herramienta admite varias modalidades de contenido [11], y en este trabajo intervienen dos con papeles distintos. Los proyectos de ejemplo publicados por el fabricante están generados con *MCC Classic* y se emplean como corpus de contraste; los proyectos propios, de los que sale el patrón de referencia, se generan con *MCC Melody*, la única modalidad que ofrece la extensión de MPLAB para Visual Studio Code. La distinción no es accesoria: cada modalidad genera código con una estructura distinta, de modo que la comparación estructural de un

archivo generado solo tiene sentido dentro de la misma modalidad. Por esa razón la fidelidad se contrasta sobre los valores escritos en los registros de configuración, que no dependen de la modalidad que los produjo.

Esta separación entre especificación y código resulta de interés metodológico: cada proyecto generado con MCC constituye un par formado por una especificación declarativa y el código que el fabricante considera su realización correcta. Ese par ofrece un patrón de referencia natural frente al cual contrastar el código que produzca un sistema automático.

#### 2.2.4. Criterios de eficiencia

La evaluación de un firmware no se agota en su corrección funcional. En un dispositivo con 128 KB de memoria de programa y algo menos de 3,3 KB utilizables de memoria de datos, el consumo de recursos es una restricción de diseño. Se adoptan como métricas los bytes ocupados en cada una de esas memorias, tomados del informe del compilador, lo que permite comparar realizaciones distintas de una misma funcionalidad.

### 2.3. Modelos de lenguaje de gran escala y generación de código

Un modelo de lenguaje de gran escala, designado habitualmente por sus siglas en inglés LLM (*Large Language Model*), es una red neuronal entrenada para estimar la probabilidad del siguiente elemento de una secuencia de texto. Su aplicación a la generación de código explota el hecho de que un programa es, formalmente, una secuencia de símbolos con regularidades estadísticas.

#### 2.3.1. La arquitectura Transformer

Los modelos actuales derivan de la arquitectura Transformer, que prescinde de la recurrencia y de la convolución y basa el procesamiento exclusivamente en mecanismos de atención [12]. La atención pondera la relevancia de cada elemento de la secuencia respecto de los demás, lo que permite capturar dependencias entre posiciones alejadas sin recorrer las intermedias.

Su ventaja decisiva es la paralelización: al no existir dependencia secuencial entre posiciones, el entrenamiento aprovecha el cómputo paralelo, lo que hizo viable escalar tanto el tamaño de los modelos como el de los corpus.

Formalmente, la atención opera sobre tres proyecciones de la secuencia de entrada —consultas  $Q$ , claves  $K$  y valores  $V$ — y se calcula como

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (2.1)$$

donde  $d_k$  es la dimensión de las claves y el factor  $1/\sqrt{d_k}$  evita que el producto escalar crezca hasta saturar la función exponencial [12]. El modelo aplica varias de estas operaciones en paralelo, cada una con proyecciones distintas, lo que le permite atender simultáneamente a relaciones de naturaleza diferente.

### 2.3.2. Tokenización y ventana de contexto

El texto no se procesa carácter a carácter sino en unidades denominadas *tokens*, obtenidas por un algoritmo de segmentación subléxica. Un modelo solo puede atender a un número acotado de *tokens* simultáneamente: es la llamada ventana de contexto.

Esta limitación es central para este trabajo. El manual del dispositivo supera las setecientas páginas, de modo que no cabe íntegro en la ventana de contexto de los modelos de uso corriente. Aun cuando cupiera, la evidencia experimental muestra que la capacidad de un modelo para utilizar la información degrada con la longitud del contexto, y que los elementos situados en posiciones centrales se aprovechan peor que los de los extremos [7].

La segmentación subléxica tiene una consecuencia directa en este dominio. Un identificador como `ANSELDbits` no constituye una unidad para el modelo, sino que se fragmenta en varias piezas según su frecuencia en el corpus. Cuanto más raro sea el término, en más piezas se divide y menor es la evidencia estadística de que dispone el modelo para reproducirlo con exactitud.

### 2.3.3. Estrategias de decodificación

Un modelo no emite un símbolo determinado sino una distribución de probabilidad sobre el vocabulario, y la estrategia de decodificación decide cómo obtener de ella la salida. La selección voraz toma en cada paso el elemento más probable y produce resultados reproducibles. El muestreo introduce aleatoriedad controlada por un parámetro de temperatura: valores bajos concentran la probabilidad en los candidatos dominantes y valores altos la reparten.

Esta elección afecta a la evaluación. Una temperatura no nula hace que dos ejecuciones con la misma entrada produzcan salidas distintas, de modo que cualquier métrica debe calcularse sobre varias muestras y acompañarse del parámetro empleado. La métrica *pass@k* presupone precisamente ese muestreo repetido [2].

#### 2.3.4. Generación de código y su evaluación

El trabajo que estableció el marco de evaluación hoy dominante presentó un modelo entrenado específicamente sobre código junto con un conjunto de problemas de programación y una métrica asociada [2]. Esa métrica, denominada *pass@k*, estima la probabilidad de que al menos una de  $k$  muestras generadas supere las pruebas del problema.

Su interés radica en que evalúa el comportamiento del código, no su parecido con una solución de referencia. Un programa puede resolver correctamente un problema con una redacción muy distinta de la esperada, y las métricas basadas en similitud textual lo penalizarían.

#### 2.3.5. Instrucciones de sistema

Además de la consulta, el modelo recibe un texto que fija su comportamiento: el papel que adopta, el formato de la respuesta y las restricciones que debe respetar. En un sistema de generación de firmware, ese texto es el lugar donde se imponen la nomenclatura exacta de los registros, el estilo de código del compilador y la obligación de citar la sección del manual que respalda cada decisión. Su redacción constituye, por tanto, una decisión de diseño y no un detalle de implementación.

#### 2.3.6. Limitaciones

Tres limitaciones condicionan el empleo directo de estos modelos en el dominio que ocupa a este trabajo.

La primera es la generación de contenido incorrecto con apariencia plausible. Un modelo puede producir nombres de registro inexistentes o campos de bits con anchura equivocada, y presentarlos con la misma seguridad que los correctos.

La segunda es el corte de conocimiento: el modelo solo dispone de lo que estaba en su corpus de entrenamiento, sin acceso a documentación posterior ni a erratas.

La tercera, específica del dominio, es la escasez de vocabulario especializado. Los nombres de registro, los prefijos de fabricante y las constantes de hardware aparecen con muy baja frecuencia en corpus generalistas, de modo que el modelo dispone de poca evidencia estadística sobre ellos.

### 2.3.7. Antecedentes en sistemas embebidos

La literatura reciente ha caracterizado experimentalmente estas capacidades y límites. Un estudio sobre desarrollo y depuración de sistemas empotrados concluye que los modelos actuales alcanzan un grado notable de razonamiento entre dominios, generando controladores a nivel de registro y código de comunicación funcional, y que incluso cuando el código resultante no es correcto el razonamiento que lo acompaña resulta de utilidad [3]. El mismo trabajo observa que parte del conocimiento del modelo sobre el hardware procede de la presencia de hojas de datos en su corpus de entrenamiento, y que suministrarle una representación legible de los registros relevantes mejora sustancialmente el resultado.

Esta observación es precisamente la hipótesis de la que parte el presente proyecto. En la misma línea, un trabajo posterior aplica recuperación de información a la generación de capas de abstracción de hardware para microcontroladores de otro fabricante, integrándola en el flujo de desarrollo existente [5]. Ambos antecedentes delimitan el espacio en que se sitúa esta pasantía: la técnica está validada, y lo que se aporta es su realización sobre la familia PIC18-Q10 y su cadena de herramientas, con un ciclo de validación basado en compilación.

## 2.4. Generación aumentada por recuperación

La generación aumentada por recuperación, conocida por sus siglas en inglés RAG (*Retrieval-Augmented Generation*), combina un modelo de lenguaje con un sistema de recuperación de información sobre un corpus externo [4]. En lugar de responder únicamente con el conocimiento almacenado en sus parámetros, el modelo recibe junto a la consulta un conjunto de fragmentos recuperados del corpus y genera a partir de ellos.

### 2.4.1. Fundamento

La formulación original distingue dos tipos de memoria. La memoria paramétrica es el conocimiento codificado en los pesos durante el entrenamiento: extenso pero difuso, imposible de inspeccionar y costoso de actualizar. La memoria no paramétrica es un índice sobre documentos, consultable en tiempo de inferencia y modificable sin reentrenar [4].

Para el problema que ocupa a este trabajo, esa distinción resuelve tres dificultades simultáneamente. La documentación del dispositivo puede actualizarse sin tocar el modelo; la respuesta queda anclada a un fragmento concreto y, por tanto, es verificable; y la escasez de vocabulario especializado en el corpus de entrenamiento deja de ser determinante, porque los nombres de registro llegan al modelo en el propio contexto.

### 2.4.2. Arquitectura

Un sistema RAG opera en tres etapas, representadas en la figura 2.2. La indexación se ejecuta una sola vez, mientras que la recuperación y la generación se ejecutan en cada consulta.

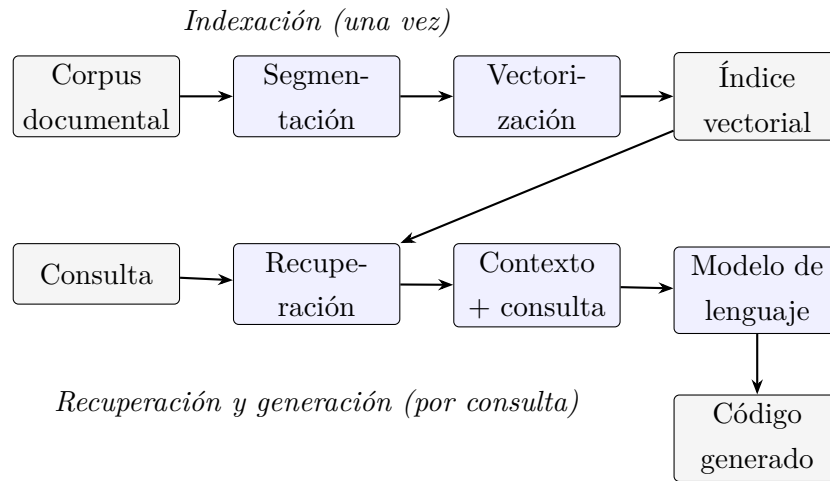


Figura 2.2. Etapas de un sistema de generación aumentada por recuperación

### 2.4.3. Extracción de documentación técnica

Antes de segmentar hay que convertir el documento a texto, y en formatos paginados esa conversión no es neutra. Un extractor orientado a prosa recorre el contenido en orden de lectura y produce una secuencia plana, con lo que la información dispuesta en dos dimensiones se pierde.

El efecto sobre el material de este proyecto es directo. Un mapa de registros presenta en columnas la dirección, el nombre y los campos de bits; leído como texto plano, esas columnas se entremezclan y el resultado es inservible. Conservar la alineación mediante las opciones adecuadas del extractor mantiene la tabla legible, pero no todos los elementos sobreviven: los mapas de memoria, los diagramas de bloques y los esquemas de pines son figuras, y de ellos no queda nada. Tampoco sobrevive el sombreado con que algunas tablas distinguen a qué dispositivo aplica cada fila.

De ahí que la elección de la herramienta de extracción no sea un detalle de implementación sino una decisión que condiciona la calidad de todo el corpus. Las alternativas van desde los extractores de texto tradicionales hasta los sistemas que reconstruyen la estructura del documento y representan las tablas como tales. Cuando un dato depende de una figura, la única vía es interpretarla como imagen o transcribirla manualmente.

#### 2.4.4. Segmentación del corpus

Los documentos no se indexan enteros sino divididos en fragmentos. El tamaño del fragmento establece un compromiso: uno demasiado extenso diluye la señal de relevancia e introduce ruido en el contexto; uno demasiado breve pierde la información necesaria para interpretarlo.

En documentación técnica esta decisión adquiere una dificultad añadida. Una tabla de registros separada de su encabezado pierde el significado de sus columnas, y una cifra separada de la nota al pie que la matiza puede inducir a error. El manual del dispositivo contiene ejemplos de ambas situaciones [1]. La estrategia de segmentación debe, por tanto, preservar la unidad entre una tabla, su título y sus notas.

Se distinguen tres familias de estrategias. La segmentación por **tamaño fijo** divide el texto en bloques de longitud constante, habitualmente con solapamiento entre bloques consecutivos para no partir una idea por la mitad; es simple pero ignora la estructura del documento. La segmentación **por estructura** aprovecha la jerarquía de secciones y subsecciones, de modo que cada fragmento coincide con una unidad lógica. La segmentación **semántica** decide los cortes según la afinidad entre pasajes contiguos.

Para documentación técnica, la segunda resulta la más adecuada, porque la jerarquía del manual ya refleja la organización del conocimiento. Requiere, eso sí, una extracción del documento que conserve esa jerarquía, lo que en formatos paginados no es inmediato.

#### 2.4.5. Metadatos y filtrado

Cada fragmento puede acompañarse de atributos que no participan en la comparación vectorial pero permiten restringir la búsqueda: el documento de origen, la sección, la página, el periférico tratado o el dispositivo al que aplica.

Este último resulta indispensable en el caso que ocupa a este trabajo. Dado que el manual describe simultáneamente dos dispositivos de la familia [1], un fragmento recuperado sin filtrar por dispositivo puede contener cifras que no corresponden al integrado empleado. El filtrado por metadatos convierte esa distinción en una restricción explícita de la consulta en lugar de dejarla al criterio del modelo.

#### 2.4.6. Representación vectorial y búsqueda densa

La recuperación densa representa fragmentos y consultas como vectores en un espacio de dimensión fija, de modo que la proximidad geométrica aproxime la afinidad semántica [13]. La consulta se vectoriza con el mismo procedimiento y se recuperan los  $k$  fragmentos más

próximos según una medida de similitud, habitualmente el coseno del ángulo entre vectores.

La obtención de representaciones adecuadas para comparar textos completos exigió adaptaciones específicas sobre los modelos de lenguaje preexistentes, ya que estos no producían de forma nativa vectores comparables mediante similitud [14].

La proximidad entre vectores se cuantifica habitualmente mediante la similitud del coseno, que para dos vectores  $\mathbf{u}$  y  $\mathbf{v}$  se define como

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}. \quad (2.2)$$

Su interés reside en que atiende únicamente a la orientación de los vectores y no a su magnitud, de modo que un fragmento extenso y otro breve que traten el mismo asunto resultan igualmente próximos a la consulta.

La ventaja de este enfoque es que reconoce equivalencias semánticas sin coincidencia literal. Su limitación aparece precisamente en el dominio de este trabajo: un identificador como TRISD o PPS1WAY carece de sinónimos, y lo que interesa de él es su aparición exacta.

#### 2.4.7. Recuperación cruzada entre idiomas

La documentación del fabricante está escrita en inglés y las peticiones al sistema se formulan en español. La recuperación es, por tanto, cruzada entre idiomas por diseño, y esa condición impone un requisito sobre la representación vectorial que conviene establecer antes de elegir el modelo que la produce.

Un modelo entrenado sobre un solo idioma no dispone de ningún mecanismo que acerque una expresión a su traducción. La consulta «*configurar el conversor analógico-digital*» y el fragmento «*ADC with Computation*» designan lo mismo, pero un codificador monolingüe los sitúa en regiones distintas del espacio: la proximidad geométrica que la ecuación (2.2) mide deja de aproximar la afinidad semántica en cuanto los dos textos no comparten idioma.

Los modelos multilingües alineados resuelven esa separación construyendo un espacio común. El procedimiento de destilación de conocimiento propuesto por Reimers y Gurevych entrena un modelo alumno para que sitúe una frase traducida *en el mismo lugar* que ocupa la original en el espacio del modelo maestro, de modo que la equivalencia entre idiomas queda codificada en la geometría y no delegada a una traducción previa [15]. El alineamiento entre idiomas admite además otras familias de métodos, y su alcance y sus límites han sido sistematizados recientemente [16].

La consecuencia práctica para este trabajo no es la pérdida de calidad en sí, sino su

carácter silencioso. Un recuperador mal alineado no falla: devuelve los  $k$  fragmentos solicitados, formalmente correctos, simplemente peores. No hay excepción, ni aviso, ni indicio en la respuesta generada de que el contexto entregado no era el pertinente. Por eso el alineamiento entre idiomas no puede suponerse a partir de la descripción del modelo y debe verificarse experimentalmente sobre el corpus del problema, lo que se aborda en el capítulo siguiente.

#### 2.4.8. Búsqueda léxica y estrategias híbridas

Frente a la anterior, la recuperación léxica puntúa los fragmentos según la coincidencia de términos, ponderando su frecuencia en el documento y su rareza en la colección. La formulación de referencia es BM25, dentro del marco probabilístico de relevancia [17].

Las dos aproximaciones son complementarias: la densa capta la intención de la consulta y la léxica garantiza que un identificador exacto no se pierda. Las estrategias híbridas combinan ambas puntuaciones, lo que resulta especialmente pertinente cuando el corpus está poblado de nomenclatura literal.

La combinación no es inmediata, porque las dos puntuaciones no comparten escala. Un procedimiento habitual prescinde de los valores y opera sobre las posiciones: la fusión recíproca de rangos asigna a cada fragmento una puntuación

$$s(d) = \sum_i \frac{1}{c + r_i(d)}, \quad (2.3)$$

donde  $r_i(d)$  es la posición del fragmento  $d$  en la lista  $i$  y  $c$  una constante que atenúa el peso de las primeras posiciones. Al depender solo del orden, evita tener que normalizar puntuaciones heterogéneas.

#### 2.4.9. Índices aproximados

Comparar la consulta con todos los vectores del índice resulta inviable cuando este crece. Los índices de vecinos aproximados sacrifican la garantía de encontrar el óptimo exacto a cambio de una reducción drástica del tiempo de búsqueda. La estructura más extendida organiza los vectores en un grafo jerárquico navegable, en el que la búsqueda desciende por capas de densidad creciente [18].

#### 2.4.10. Bases de datos vectoriales

El almacenamiento de los vectores y sus metadatos, junto con el índice que los recorre, corresponde a una base de datos vectorial. Frente a una realización propia sobre una biblioteca

de índices, estos sistemas aportan persistencia, filtrado por metadatos integrado en la búsqueda, actualización incremental del índice y control de versiones de la colección.

Esa integración importa: filtrar después de recuperar puede dejar menos de  $k$  resultados válidos, mientras que hacerlo durante el recorrido del índice garantiza que los  $k$  devueltos cumplen la restricción.

#### **2.4.11. Reordenamiento**

La recuperación inicial prioriza el recorrido rápido de un índice extenso, lo que obliga a comparar consulta y fragmento por separado. Una segunda etapa puede reordenar un conjunto reducido de candidatos evaluándolos conjuntamente con la consulta, lo que mejora la precisión a costa de un cómputo mayor [19]. Existen formulaciones intermedias que retrasan la interacción entre ambos textos para conservar parte de esa precisión sin renunciar a la eficiencia [20].

#### **2.4.12. Construcción del contexto**

Los fragmentos recuperados se incorporan al contexto junto con la consulta y las instrucciones de sistema. Dado que el aprovechamiento del contexto no es uniforme a lo largo de su extensión [7], tanto el número de fragmentos como su orden constituyen decisiones de diseño con efecto medible sobre el resultado.

#### **2.4.13. Recuperación frente a ajuste fino**

La alternativa a la recuperación es el ajuste fino, esto es, continuar el entrenamiento del modelo sobre el corpus del dominio para incorporarlo a sus parámetros. La tabla 2.3 contrasta ambas opciones en los aspectos determinantes para este proyecto.

Tabla 2.3. Recuperación aumentada frente a ajuste fino

| Aspecto                        | Recuperación aumentada             | Ajuste fino                    |
|--------------------------------|------------------------------------|--------------------------------|
| Actualización del conocimiento | Reindexar el corpus                | Reentrenar el modelo           |
| Trazabilidad de la respuesta   | El fragmento citado es verificable | El origen no es inspeccionable |
| Coste computacional            | Indexación y búsqueda              | Entrenamiento sobre acelerador |
| Volumen de datos exigido       | Los documentos disponibles         | Muchos ejemplos etiquetados    |
| Jerarquía entre fuentes        | Se impone al recuperar             | Se diluye en los pesos         |

La última fila resulta determinante en este dominio. La errata de silicio corrige al manual y debe prevalecer sobre él [6]. Un sistema de recuperación puede imponer esa jerarquía de forma explícita; un modelo ajustado sobre ambos documentos los mezcla en sus parámetros sin conservar la relación de precedencia. A ello se añade que el volumen documental disponible —un manual, una errata y un conjunto de notas de aplicación— es adecuado para construir un índice, pero insuficiente para un entrenamiento.

#### 2.4.14. Variantes

Sobre la formulación básica se han propuesto extensiones. Algunas incorporan al modelo la decisión de cuándo recuperar y la evaluación crítica de lo recuperado [21]. Otras añaden una etapa correctiva que detecta recuperaciones de baja calidad y actúa en consecuencia [22]. Una tercera línea construye un grafo de entidades y relaciones sobre el corpus para responder consultas que exigen agregar información dispersa [23].

### 2.5. Exposición del sistema mediante el Model Context Protocol

Un sistema de recuperación resuelve qué fragmentos entregar al modelo, pero no cómo hacérselos llegar. Históricamente esa unión se resolvía de forma particular para cada pareja de aplicación y fuente de datos, lo que multiplica las integraciones:  $N$  aplicaciones frente a  $M$  fuentes exigen  $N \times M$  adaptadores. El *Model Context Protocol* (MCP) es una especificación abierta que normaliza esa frontera, de modo que cualquier cliente compatible pueda consumir cualquier servidor compatible [24].

Para este trabajo la elección no es accesorio. El sistema de recuperación construido sobre

el corpus del PIC18F47Q10 se expone como un servidor MCP, y con ello queda disponible para cualquier asistente que hable el protocolo sin reescribir la integración. Esa separación entre el recuperador y el modelo que lo consume es además lo que permite comparar el desempeño con y sin recuperación, que es el diseño experimental descrito en la sección siguiente.

### 2.5.1. Arquitectura y capas

El protocolo distingue tres participantes. El *host* es la aplicación que coordina —el asistente que el usuario maneja—; el *cliente* es el componente que mantiene la conexión, y el *servidor* es el proceso que expone las capacidades. La especificación fija una relación uno a uno entre cliente y servidor: el host instancia un cliente por cada servidor al que se conecta, lo que aísla los fallos y delimita los permisos de cada fuente [24].

Sobre esa disposición se definen dos capas. La capa de datos establece el intercambio mediante JSON-RPC 2.0 y comprende el ciclo de vida de la conexión, el descubrimiento de capacidades y las notificaciones. La capa de transporte determina el canal físico —entrada y salida estándar para servidores locales, HTTP para los remotos— y es independiente de la anterior, de manera que un mismo servidor puede exponerse por cualquiera de las dos sin modificar su lógica [24].

### 2.5.2. Primitivas y control

El servidor expone tres tipos de capacidad, y lo que las distingue no es su contenido sino quién decide invocarlas. Las *herramientas* (*tools*) las elige el modelo durante la generación; los *recursos* (*resources*) los selecciona la aplicación anfitriona; y las *plantillas de instrucciones* (*prompts*) las invoca el usuario de forma explícita [24].

Esa asignación de control importa para el diseño del recuperador. Una consulta al corpus del microcontrolador debe poder originarse en el propio modelo cuando detecta que le falta información sobre un registro, lo que la sitúa naturalmente como herramienta y no como recurso.

### 2.5.3. Estado de la especificación

MCP es una especificación viva y sus revisiones se identifican por fecha. La versión vigente durante el desarrollo de este trabajo es 2026-07-28 [24]. Conviene señalar que dos primitivas del cliente presentes en revisiones anteriores —el muestreo (*sampling*) y el registro (*logging*)— quedaron obsoletas en esa revisión, circunstancia que invalida buena parte del material divulgativo publicado con anterioridad y que obliga a contrastar cualquier ejemplo

contra la especificación vigente.

## 2.6. Evaluación de sistemas de generación de código

La evaluación debe abarcar dos componentes con modos de fallo distintos: el recuperador, que puede devolver fragmentos irrelevantes, y el generador, que puede producir código incorrecto a partir de fragmentos adecuados. Medirlos por separado permite atribuir los fallos.

### 2.6.1. Métricas sobre el código generado

La métrica de referencia en generación de código estima la probabilidad de que al menos una de  $k$  muestras supere un conjunto de pruebas [2]. Si de un problema se generan  $n$  muestras de las que  $c$  resultan correctas, el estimador insesgado es

$$pass@k = \mathbb{E} \left[ 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right], \quad (2.4)$$

expresión que evita la varianza elevada que tendría el cálculo directo sobre  $k$  muestras [2]. Su aplicación a firmware presenta una particularidad: ejecutar el código exige el dispositivo o un simulador, lo que encarece la evaluación.

Por ello se adopta como criterio primario la **tasa de compilación exitosa**: la proporción de generaciones que el compilador acepta sin errores. Es un criterio necesario aunque no suficiente —un programa puede compilar y no funcionar—, pero resulta objetivo, automatizable y sensible a los errores característicos del dominio, como un nombre de registro inexistente.

A ella se añaden dos criterios complementarios. La **fidelidad a la configuración de referencia** compara, registro a registro, el código generado con el que produce la herramienta del fabricante para la misma especificación. La **eficiencia en recursos** contrasta el consumo de memoria de programa y de datos frente a los umbrales establecidos.

### 2.6.2. Evaluación del recuperador

El recuperador se evalúa con las métricas propias de la recuperación de información, sobre un conjunto de consultas con su respuesta esperada. La cobertura entre los  $k$  primeros resultados mide si el fragmento pertinente llegó a recuperarse; el rango recíproco medio pondera además la posición en que lo hizo.

Medir esta etapa de forma aislada es metodológicamente necesario: si el fragmento correcto no se recupera, ninguna mejora en el generador puede compensarlo, y una evaluación conjunta

no distinguiría ambas causas. Existen marcos que automatizan la valoración conjunta del sistema [25], útiles como complemento pero no como sustituto de la medida por etapas.

### 2.6.3. Validación automática con realimentación

El compilador no solo dictamina si el código es aceptable: emite diagnósticos que localizan el error y lo describen. Esa información puede reinyectarse al modelo para que corrija su propia salida, según el ciclo de la figura 2.3.

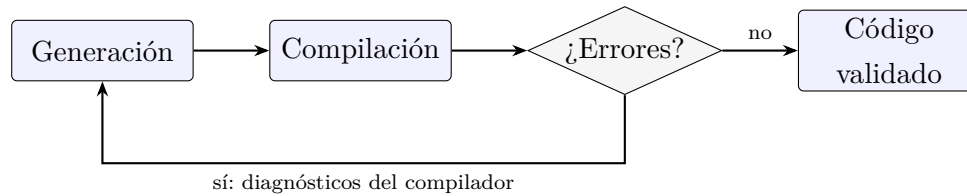


Figura 2.3. Ciclo de validación con realimentación de errores

El ciclo exige fijar dos parámetros: el número máximo de iteraciones antes de declarar el fallo y el criterio de convergencia. Ambos condicionan los resultados y deben declararse explícitamente al presentarlos, ya que la tasa de éxito con corrección iterativa no es comparable con la obtenida en un solo intento.

### 2.6.4. Diseño experimental

La afirmación central de este trabajo es que la recuperación mejora la calidad del código generado. Contrastarla exige un experimento de ablación en el que la presencia del sistema de recuperación sea la única variable: mismo modelo, mismas consultas, mismas instrucciones de sistema y mismos criterios de medida. Cualquier otra diferencia entre ambas condiciones comprometería la atribución del efecto observado.

Dos precauciones de orden estadístico acompañan a ese diseño. La primera es el tamaño del conjunto de consultas: una diferencia entre condiciones solo resulta interpretable si el número de casos permite distinguirla de la variabilidad propia del muestreo. La segunda es que, con temperatura no nula, cada consulta debe repetirse varias veces y las métricas calcularse sobre el conjunto, declarando tanto el número de repeticiones como el valor del parámetro.

La tabla 2.4 resume el conjunto de medidas y el componente que evalúa cada una.

Tabla 2.4. Métricas de evaluación y componente evaluado

| Métrica             | Componente  | Qué mide                                           |
|---------------------|-------------|----------------------------------------------------|
| <i>recall@k</i>     | Recuperador | Si el fragmento pertinente se recupera             |
| MRR                 | Recuperador | En qué posición lo hace                            |
| Tasa de compilación | Generador   | Proporción de código que el compilador acepta      |
| Fidelidad           | Generador   | Coincidencia registro a registro con la referencia |
| Eficiencia          | Generador   | Memoria de programa y de datos consumida           |
| <i>pass@k</i>       | Generador   | Probabilidad de acierto en $k$ intentos            |

## **CAPÍTULO 3**

### **METODOLOGÍA**

## **CAPÍTULO 4**

### **RESULTADOS Y DISCUSIÓN**

## Conclusiones y recomendaciones

## Referencias

- [1] Microchip Technology Inc. *PIC18F27/47Q10 Microcontrollers Data Sheet*, 2024. Documento DS40002043F, 767 páginas.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [3] Zachary Englhardt, Richard Li, Dilini Nissanka, Zhihan Zhang, Girish Narayanswamy, Joseph Breda, Xin Liu, Shwetak Patel, and Vikram Iyer. Exploring and characterizing large language models for embedded system development and debugging. *arXiv preprint arXiv:2307.03817*, 2023.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [5] Sebastian Haug, Christoph Böhm, and Daniel Mayer. Automated code generation and validation for software components of microcontrollers. *arXiv preprint arXiv:2502.18905*, 2025.
- [6] Microchip Technology Inc. *PIC18F27/47Q10 Silicon Errata and Data Sheet Clarifications*, 2024. Documento DS80000832G.
- [7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.
- [8] Microchip Technology Inc. *PIC18F2X/4XQ10 Memory Programming Specification*, 2023. Documento DS40001874H.
- [9] Microchip Technology Inc. *MPLAB XC8 C Compiler User’s Guide for PIC MCU*, 2012. Documento DS52053B.
- [10] Microchip Technology Inc. *PIC18F47Q10 Curiosity Nano Hardware User Guide*, 2020. Documento DS40002103B.

- [11] Microchip Technology Inc. *MPLAB Code Configurator (MCC) User's Guide*, 2022. Documento DS40001829F.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [13] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- [14] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [15] Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4512–4525. Association for Computational Linguistics, 2020.
- [16] Katharina Hämmerl, Jindřich Libovický, and Alexander Fraser. Understanding cross-lingual alignment—a survey. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024.
- [17] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [18] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [19] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- [20] Omar Khatatba and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [21] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.
- [23] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

- [24] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2026. Revisión del protocolo 2026-07-28; consultada el 24 de agosto de 2026.
- [25] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the ACL: System Demonstrations*, 2024.